Mejora de los modelos de calificación crediticia mediante máquinas de vectores de soporte

Entrega final — Fase 3: consolidación, despliegue y comunicación

Integrantes:

Daniela González Ortiz

Belén Shalom Dueñas Pacheco

Jhabes Esteban Rodríguez Enríquez

Curso:

Machine Learning II

Universidad Externado de Colombia

2026

Resumen

Este proyecto reproduce y adapta al contexto colombiano el método de Xu y Cheng, que propone las máquinas de vectores de soporte (SVM) para mejorar los modelos de calificación crediticia. En las entregas anteriores construimos una base simulada de 6.500 solicitantes en COP, un flujo de preprocesamiento sin fuga de datos y una validación cruzada anidada de siete modelos. En esta entrega cerramos el flujo: seleccionamos como modelo final el **SVM con kernel gaussiano (RBF)**, lo evaluamos una sola vez en prueba (ROC-AUC 0.776, igual al techo teórico de la simulación; recall de "malo" 0.866; costo 0.536 con la matriz 5:1), lo interpretamos con SHAP, dependencia parcial y LIME, y lo desplegamos en una aplicación de Streamlit que verifica los datos de entrada, aplica el pipeline completo y explica cada predicción. La interpretación muestra que el modelo recuperó la estructura de riesgo verdadera de la simulación (mismo orden de importancia, género sin efecto), y que su ventaja sobre la regresión logística es pequeña y no significativa. Cerramos con las limitaciones del trabajo, la principal de las cuales es que los datos son simulados, y con su proyección hacia *Deep Learning* y MLOps.

1. Modelo final y reproducción del método del artículo

1.1 Selección del modelo final

La Segunda entrega comparó siete modelos con validación cruzada anidada (5 folds externos × 3 internos) sobre los 5.200 clientes de entrenamiento. Para elegir el modelo final sopesamos los tres criterios de la guía (Tabla 1 y Figura 1):

- **Desempeño.** El SVM RBF es el mejor según la regla fijada antes de ver los resultados (mayor ROC-AUC medio y, en empate, menor costo): tiene además el mejor F1 y el menor costo. Su ventaja sobre la regresión logística, sin embargo, es mínima (+0.001 de AUC, $p = 0.29$). Lo confirmamos en prueba con un bootstrap pareado de 2.000 remuestreos: la diferencia de AUC es +0.002 (IC 95%: −0.004 a +0.008) y la de costo −0.003 (IC 95%: −0.031 a +0.023). Los dos modelos toman la misma decisión en el 94% de los clientes y sus puntajes tienen una correlación de Spearman de 0.985.
- **Estabilidad.** El SVM RBF tiene la menor desviación de costo entre folds (0.016, empatado con el Random Forest, frente a 0.028 de la logística y 0.036 del Gradient Boosting) y elige los mismos hiperparámetros ($C = 1$, $\gamma = 0.01$) en 4 de los 5 folds externos. El árbol de decisión, en cambio, es inestable y claramente inferior.
- **Interpretabilidad.** Es la debilidad del SVM RBF: no tiene coeficientes. La logística, el SVM lineal y el árbol son directamente interpretables.

Tabla 1. Criterios de selección. Media ± desviación en la validación anidada (5 folds externos); el p-valor (prueba t corregida de Nadeau y Bengio) compara el ROC-AUC de cada modelo con la logística. Las dos últimas columnas son la evaluación única en prueba (1.300 clientes).

Modelo	ROC-AUC anidada	Costo 5:1 anidado	p-valor (AUC)	ROC-AUC prueba	Costo prueba	Interpretabilidad
Regresión logística	0.765 ± 0.018	0.550 ± 0.028	—	0.774	0.539	Alta (coeficientes)
SVM lineal	0.765 ± 0.017	0.552 ± 0.025	0.99	0.774	0.545	Alta (coeficientes)
SVM RBF	**0.766 ± 0.017**	**0.541 ± 0.016**	**0.29**	**0.776**	**0.536**	**Baja (requiere SHAP)**
SVM RBF + LASSO	0.765 ± 0.017	0.549 ± 0.022	0.60	0.776	0.539	Baja (requiere SHAP)
Árbol de decisión	0.708 ± 0.013	0.645 ± 0.030	0.04	0.737	0.586	Alta (reglas)
Random Forest	0.755 ± 0.011	0.578 ± 0.016	0.15	0.770	0.572	Baja
Gradient Boosting	0.762 ± 0.018	0.560 ± 0.036	0.47	0.773	0.525	Media-baja
Techo teórico P(malo | x)	0.775 ± 0.019	0.536 ± 0.033	—	0.776	0.520	—

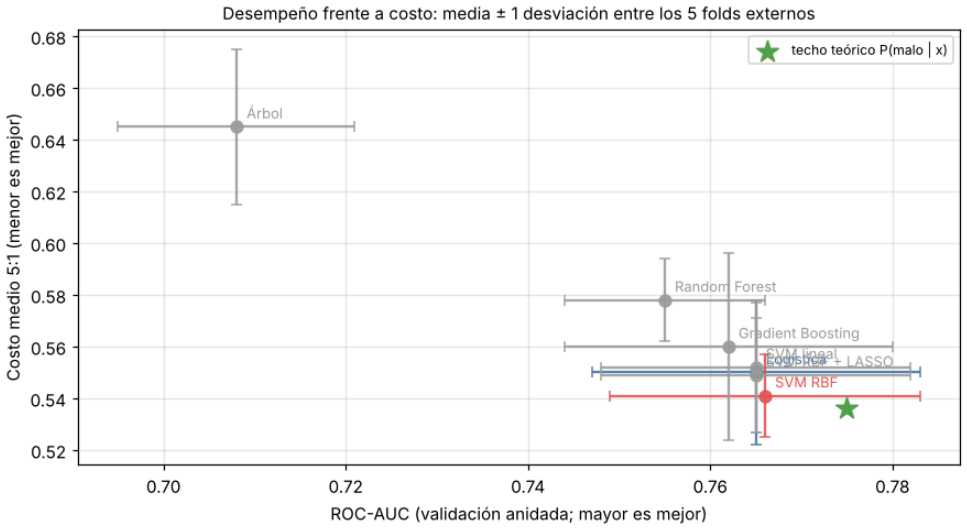


Figura 1. Desempeño frente a costo en la validación anidada (media ± 1 desviación entre folds externos). Los modelos lineales o casi lineales forman un grupo compacto junto al techo teórico (estrella verde); el árbol de decisión queda claramente por fuera.

Decisión. Elegimos el **SVM RBF** como modelo final y documentamos la **regresión logística como modelo retador**. No lo elegimos porque sea significativamente mejor, sino porque no es peor en ninguna métrica, es el más estable en costo y es el método que el proyecto se propuso reproducir: descartar el método del artículo exigiría una ventaja clara de la alternativa, y no la hay. Su desventaja en interpretabilidad la compensamos con tres herramientas que se presentan en la Sección 2: valores SHAP a nivel de las variables originales, dependencia parcial y una calibración que traduce su margen a probabilidad. Esa interpretación muestra, además, que el kernel capta la única no linealidad de la base (la forma en U de la edad), algo que la logística no puede representar. El costo de la elección es operativo: 3.645 vectores de soporte y alrededor de 1.5 segundos para explicar cada predicción. Si una entidad necesitara reglas auditables coeficiente por coeficiente, la logística ofrece el mismo desempeño estadístico y sería la alternativa natural.

1.2 Contraste con el método y los objetivos del artículo

Xu y Cheng plantean que el SVM mejora la calificación crediticia frente a los modelos tradicionales, lo evalúan sobre el German Credit y destacan como variables más relevantes el saldo de la cuenta, la duración, el historial de crédito y el monto. La Tabla 2 resume en qué medida reprodujimos cada elemento y qué cambió en la adaptación.

Tabla 2. Reproducción y adaptación del método: qué se logró y qué diferencias aparecieron.

Elemento del artículo	Resultado en el German Credit	Resultado en la base colombiana
SVM RBF con ~75% de accuracy	Reproducido: 75.5% ± 2.8%	Mejor modelo nominal; ROC-AUC 0.776 en prueba, igual al techo teórico
Ventaja del SVM sobre la logística (75% frente a 70%)	No reproducida: logística 74.8%, p = 0.76; el 70% coincide con predecir siempre "bueno"	No reproducida: +0.001 de AUC (p = 0.29); 94% de decisiones idénticas
Variables más relevantes: saldo, duración, historial y monto	Saldo y duración, las dos primeras del SVM lineal	Historial, cuenta bancaria, plazo y monto entre las seis primeras por SHAP
Selección de variables	—	LASSO conserva 30 de 32 columnas y no mejora al SVM
Evaluación por accuracy	Usada para reproducir	Reemplazada por ROC-AUC, F1, recall y costo 5:1 (desbalance de 22.5%)

En síntesis, el **desempeño del SVM se reproduce** y el método se adapta sin dificultad a la base local, pero la **conclusión central del artículo, la superioridad del SVM sobre la regresión logística, no se sostiene** cuando ambos modelos se preparan en igualdad de condiciones (variables escaladas, regularización ajustada y validación sin fuga), ni en el German Credit ni en la base colombiana. En nuestra base esto tiene además una explicación estructural: el riesgo se simuló como una función casi lineal en el logit, de modo que un modelo lineal ya está cerca del óptimo. Las diferencias de la adaptación son metodológicas, no de propósito: pesos de clase, faltantes tratados dentro del pipeline, selección por ROC-AUC, umbral de costo mínimo y validación anidada.

1.3 Configuración definitiva y evaluación en prueba

El modelo final se entrena con `src/modelo_final.py`, con el mismo procedimiento que se evaluó en la validación anidada, y se guarda en `models/modelo_final.joblib` junto con el umbral, la calibración y el perfil de los datos de entrenamiento (Tabla 3).

Tabla 3. Configuración definitiva del modelo final.

Componente	Configuración
Preprocesamiento	Saldo imputado con la mediana de su categoría; resto con mediana o moda; escalamiento estándar; one-hot (32 columnas)
Clasificador	`SVC(kernel="rbf", C=1, gamma=0.01, class_weight="balanced")`; 3.645 vectores de soporte
Hiperparámetros	Búsqueda aleatoria de 12 combinaciones en una malla 5 × 5, validación interna de 3 folds, criterio ROC-AUC
Umbral de decisión	Margen ≥ −0.492 (probabilidad ≥ 0.142): minimiza el costo 5:1 sobre puntajes fuera de fold de entrenamiento
Calibración	Platt: P(malo) = 1 / (1 + exp(−(1.118 · margen − 1.246))), ajustada con los mismos puntajes fuera de fold
Entrenamiento	5.200 clientes (80%, estratificado, semilla 42); prueba: 1.300 clientes usados una sola vez

En prueba el modelo alcanza un ROC-AUC de **0.776** (IC bootstrap 95%: 0.745-0.804), un recall de "malo" de **0.866** (0.827-0.904) y un costo medio de **0.536** (0.489-0.588), frente a 1.123 de aprobar a todos (Tabla 4). La curva ROC es prácticamente indistinguible de la del techo teórico (Figura 2, izquierda). El umbral está muy por debajo de 0.5 porque un falso negativo cuesta cinco veces más que un falso positivo: el óptimo teórico es 1/(1+5) ≈ 0.167, y el costo es casi plano entre 0.13 y 0.23, así que el umbral elegido en entrenamiento (0.142) está en una zona robusta (Figura 2, derecha).

Tabla 4. Evaluación única en prueba (1.300 clientes, 292 "malos") del modelo final, del retador y del techo teórico.

Modelo	ROC-AUC	F1 (malo)	Recall (malo)	Precisión (malo)	Costo 5:1	Aprobados
SVM RBF (modelo final)	**0.776**	**0.483**	**0.866**	**0.335**	**0.536**	**41.9%**
Regresión logística (retador)	0.774	0.488	0.842	0.343	0.539	44.8%
Techo teórico P(malo | x)	0.776	0.502	0.836	0.359	0.520	47.7%

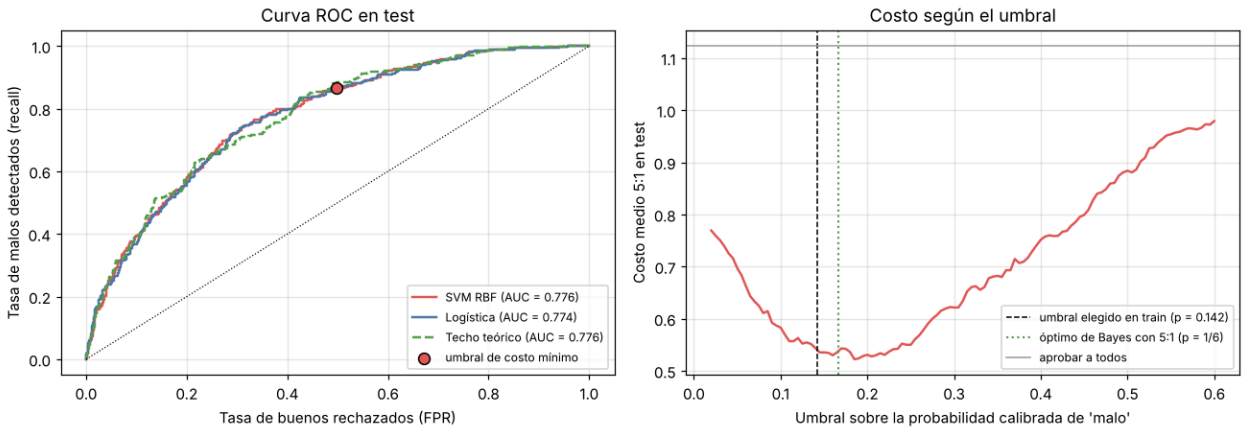


Figura 2. Izquierda: curvas ROC en prueba; el punto rojo es el umbral de costo mínimo (detecta al 87% de los malos a cambio de rechazar al 50% de los buenos). Derecha: costo medio en prueba según el umbral sobre la probabilidad calibrada.

Calibración. El SVM no produce probabilidades, y una aplicación de crédito debe mostrar una. La calibración de Platt resultó muy precisa (Figura 3): los deciles caen sobre la diagonal (error de calibración esperado de 0.015), la probabilidad media predicha (23.0%) coincide con la tasa observada (22.5%) y el Brier score (0.1440) es prácticamente el de la probabilidad verdadera (0.1435). Frente a la probabilidad verdadera de cada cliente, el error absoluto medio es de 3.8 puntos porcentuales. Como la calibración es monótona, no cambia el ROC-AUC ni las decisiones, y permite recalcular el umbral para cualquier otra matriz de costos.

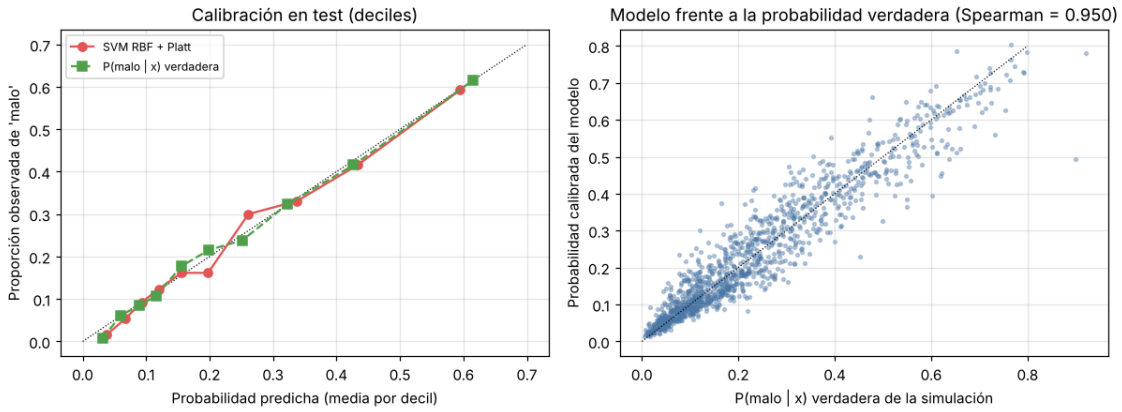


Figura 3. Izquierda: curva de calibración en prueba por deciles. Derecha: probabilidad calibrada del modelo frente a la probabilidad verdadera de la simulación para cada cliente de prueba (correlación de Spearman 0.95).

2. Interpretación de resultados

2.1 Variables más influyentes (SHAP)

Calculamos los valores SHAP con el algoritmo de permutación sobre la función completa (pipeline + SVM), tratando cada una de las 11 variables del formulario como una unidad, de modo que la explicación habla de "historial de pago" y no de cuatro columnas one-hot. Para cada cliente, el valor base más la suma de sus valores SHAP reproduce exactamente su puntaje. Usamos 400 clientes de prueba y un fondo de 50 clientes de entrenamiento. Como la base es simulada, conocemos además la contribución *verdadera* de cada variable al riesgo y podemos verificar si el modelo aprendió lo correcto, algo imposible con datos reales.

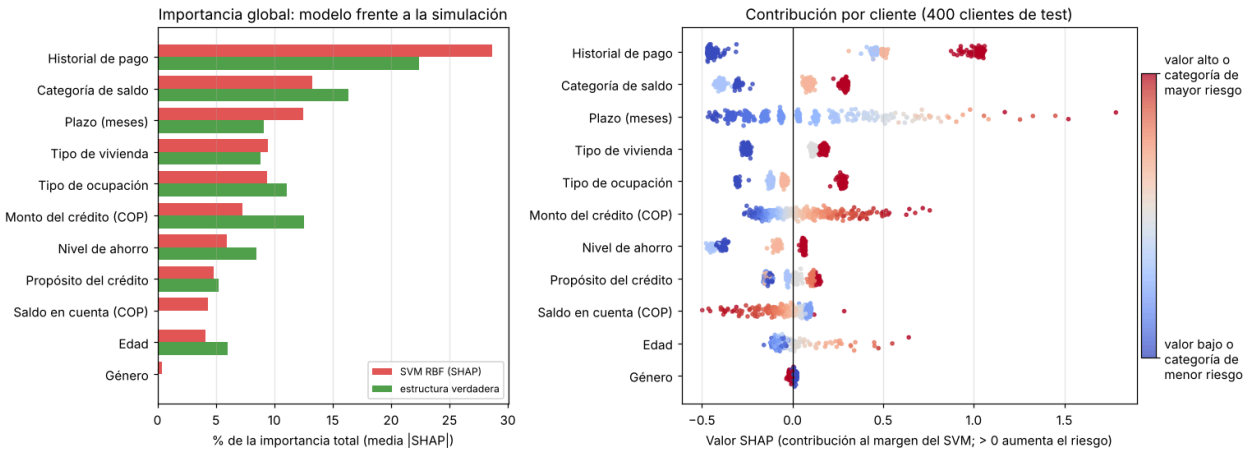


Figura 4. Izquierda: participación de cada variable en la importancia total (media del valor SHAP absoluto) del SVM y de la estructura verdadera de la simulación. Derecha: valor SHAP de cada cliente; el color indica el valor de la variable (o, en las categóricas, la categoría de mayor riesgo en rojo).

El **historial de pago** domina la decisión (29% de la importancia): una mora severa suma al puntaje más que cualquier otra característica, y estar al día es el principal factor protector. Le siguen la **cuenta bancaria** (categoría y saldo, 18%), el **plazo** (12%), la **vivienda** y la **ocupación** (9% cada una) y el **monto** (7%). Este orden coincide con el de la simulación (Spearman 0.88), y cliente por cliente las

contribuciones del modelo se correlacionan con las verdaderas por encima de 0.94 en las seis variables más importantes; promediadas por categoría, la correlación es 0.92. Las dos diferencias visibles no son errores del modelo sino una propiedad de SHAP con variables correlacionadas: el efecto de la cuenta bancaria, que en la simulación solo depende de la categoría, se reparte entre la categoría y el saldo en COP (sumadas pesan 17.6% frente a 16.4% verdadero), y parte del efecto del monto se atribuye al plazo, con el que está correlacionado (0.60). El **género** tiene un efecto prácticamente nulo (0.4% de la importancia), lo que es correcto porque no interviene en el riesgo simulado.

La comparación por categoría revela una discrepancia con consecuencias prácticas: el modelo asigna a **"sin créditos previos"** casi el mismo riesgo que a la mora leve (+0.50 en escala logit frente a +0.16 verdadero). Con solo el 9% de los clientes en esa categoría, el modelo sobreestima el riesgo del historial delgado, que es justamente el de quien pide su primer crédito.

2.2 Forma de los efectos (dependencia parcial e ICE)

La Figura 5 muestra cómo cambia el riesgo al mover una variable numérica, manteniendo las demás de cada cliente. El **plazo** tiene un efecto lineal y creciente, igual en el SVM, la logística y la estructura verdadera, y las curvas individuales (ICE) son casi paralelas: no hay interacciones relevantes. El **monto** también aumenta el riesgo, aunque ambos modelos son más planos que la verdad en montos bajos. La **edad** es el caso interesante: es la única no linealidad de la simulación (forma en U, con mínimo hacia los 39 años). El SVM la recupera parcialmente (mínimo hacia los 31 años y riesgo creciente a partir de los 50), mientras que la logística, lineal en la edad, solo puede trazar una recta. Es el único lugar donde el kernel RBF aporta algo que un modelo lineal no puede representar, y explica parte de su pequeña ventaja.

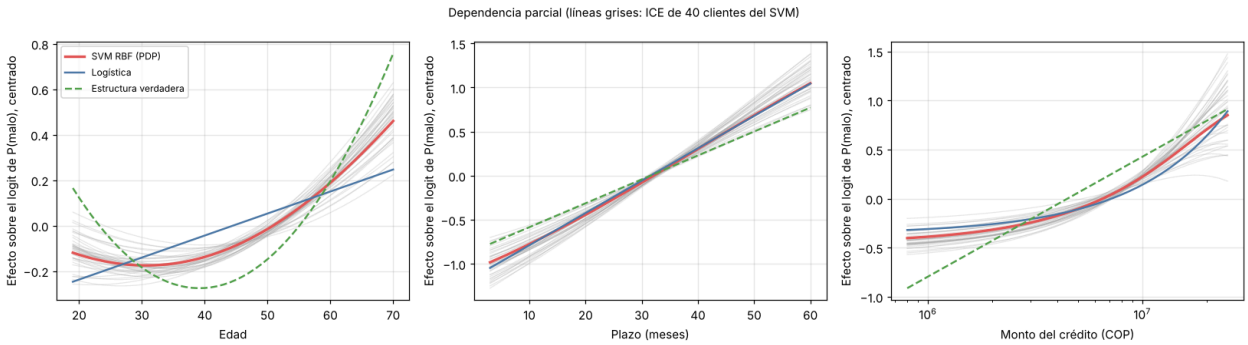


Figura 5. Dependencia parcial en escala logit, centrada: SVM RBF (rojo, con las curvas ICE de 40 clientes en gris), regresión logística (azul) y estructura verdadera de la simulación (verde, atenuada por el ruido latente).

2.3 Explicaciones individuales y tipos de error

La Figura 6 explica tres clientes de prueba con SHAP y con LIME, dos técnicas con supuestos distintos. Coinciden en la variable principal en los tres casos y en 7 de las 9 variables de los tres primeros puestos, lo que da confianza en la explicación. El **malo detectado** (A) se rechaza por mora severa y un monto de 13.1 millones. El **malo aprobado** (B), el error más caro, está al día y tiene vivienda propia, factores que pesan más que no tener cuenta; su probabilidad verdadera era 0.17, casi en el umbral. El **bueno rechazado** (C) tiene mora severa; su probabilidad verdadera era 0.40, así que el modelo

acertó al considerarlo riesgoso y que pagara es parte del azar irreducible. La aplicación usa SHAP porque LIME depende del muestreo aleatorio y de cómo discretiza las variables numéricas.

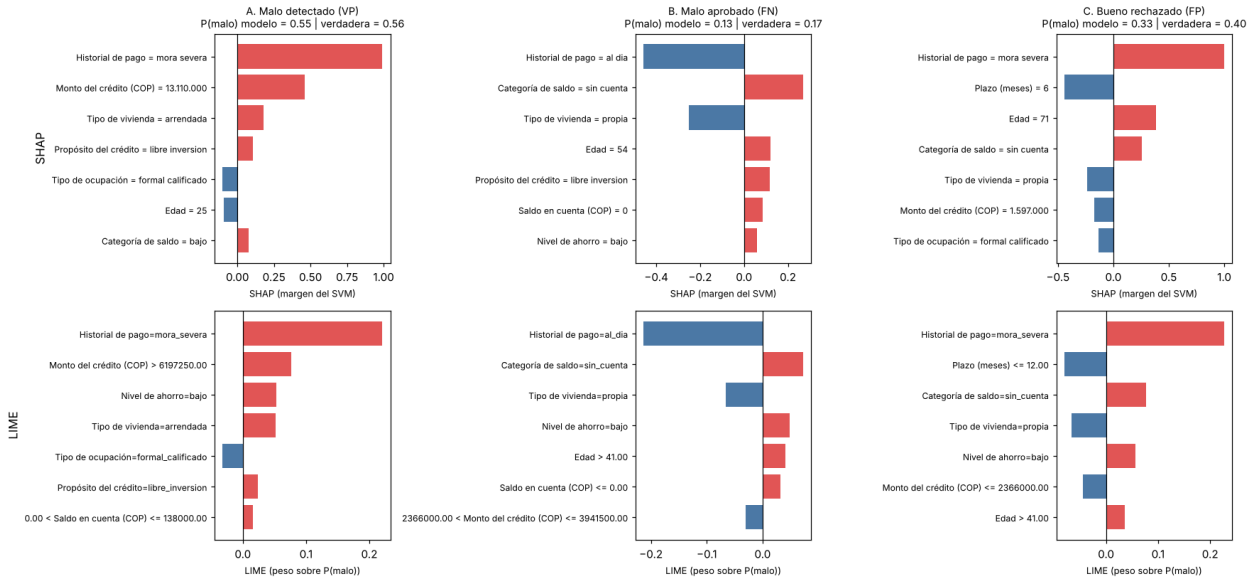


Figura 6. Explicación de tres clientes de prueba: SHAP sobre el margen del SVM (arriba) y LIME sobre la probabilidad calibrada (abajo). Rojo: aumenta el riesgo; azul: lo reduce.

Tipos de error. Con el umbral 5:1, el modelo detecta a 253 de los 292 malos de prueba y rechaza a 502 de los 1.008 buenos. Los buenos rechazados aportan el 72% del costo y los malos aprobados el 28%: la matriz de costos hace que convenga rechazar ante la duda. Como mostró la Segunda entrega, los malos aprobados son en su mayoría clientes al día con créditos cortos, y el 85% de ellos también los aprobaría la regla óptima con la estructura verdadera: son errores que no se corrigen con un algoritmo más complejo sino con información nueva.

Implicaciones prácticas. La política que induce el modelo tiene costos de inclusión financiera que una entidad debe decidir de forma explícita. Aprueba al 69% de los solicitantes al día, pero solo al 30% de los no bancarizados, al 22% de quienes no tienen créditos previos y al 4% de quienes tienen mora severa; entre los buenos clientes sin historial previo, rechaza al 74%. Respecto a la equidad, el modelo no usa el género y la probabilidad media predicha es igual para hombres y mujeres (0.229 y 0.231), en línea con su tasa real de malos; el recall también es similar (0.86 y 0.87). La tasa de rechazo de buenas clientes es algo mayor (53% frente a 46%) por la composición de las demás variables en la muestra, una diferencia que convendría monitorear. En un despliegue real retiraríamos el género del modelo.

La matriz de costos es una decisión de negocio. La relación 5:1 proviene del German Credit, no de una entidad colombiana. Con la probabilidad calibrada, el umbral óptimo para una relación r:1 es $1/(1+r)$, lo que permite ver cómo cambiaría la política (Tabla 5): pasar de 5:1 a 3:1 aprobaría al 63% de los solicitantes en lugar del 48%, con una mora de los aprobados de 10.7% en lugar de 8.2%.

Tabla 5. Política del modelo en prueba según la relación de costos entre un malo aprobado y un bueno rechazado (umbral óptimo sobre la probabilidad calibrada).

Costo FN : FP	Umbral	Aprobados	Recall (malo)	Buenos rechazados	Mora de los aprobados
1:1	0.500	90.7%	0.247	4.9%	18.7%
2:1	0.333	74.5%	0.534	17.5%	14.0%
3:1	0.250	63.2%	0.699	27.3%	10.7%
5:1	**0.167**	**47.7%**	**0.825**	**43.6%**	**8.2%**
10:1	0.091	24.2%	0.955	70.1%	4.1%

El umbral desplegado (0.142) se eligió minimizando el costo en entrenamiento; es algo más conservador que el óptimo teórico 1/6 y aprueba al 41.9% en prueba.

2.4 Alcance de las conclusiones

Las conclusiones sobre el modelo son sólidas dentro de la base: el SVM recupera la estructura verdadera, está bien calibrado y opera cerca del máximo alcanzable. Pero como el riesgo se simuló como función de los mismos predictores, estos resultados miden qué tan bien un modelo recupera una estructura diseñada, no su desempeño con clientes reales. Las afirmaciones sobre qué variables importan en Colombia (por ejemplo, que no tener cuenta aumenta el riesgo) son supuestos de la simulación, no hallazgos empíricos. Lo que sí es trasladable es el método: el flujo sin fuga, la validación anidada, el umbral de costo mínimo, la calibración, la verificación de los datos de uso y la forma de explicar las predicciones.

3. Despliegue básico

Implementamos una aplicación de **Streamlit** (`app/app.py`) que carga el artefacto del modelo final y, para cada solicitud, sigue este flujo: (1) verificar los datos frente al perfil de entrenamiento; (2) aplicar el pipeline completo (imputación, escalamiento, one-hot y SVM) para obtener el puntaje; (3) convertirlo en probabilidad con la calibración de Platt; (4) decidir con el umbral de costo mínimo; y (5) explicar la predicción con SHAP en lenguaje del formulario (Figura 7). La aplicación tiene tres pestañas: una solicitud individual, un lote en CSV (con descarga de resultados y explicación de cualquier fila) y una ficha del modelo con su propósito y sus límites de uso.

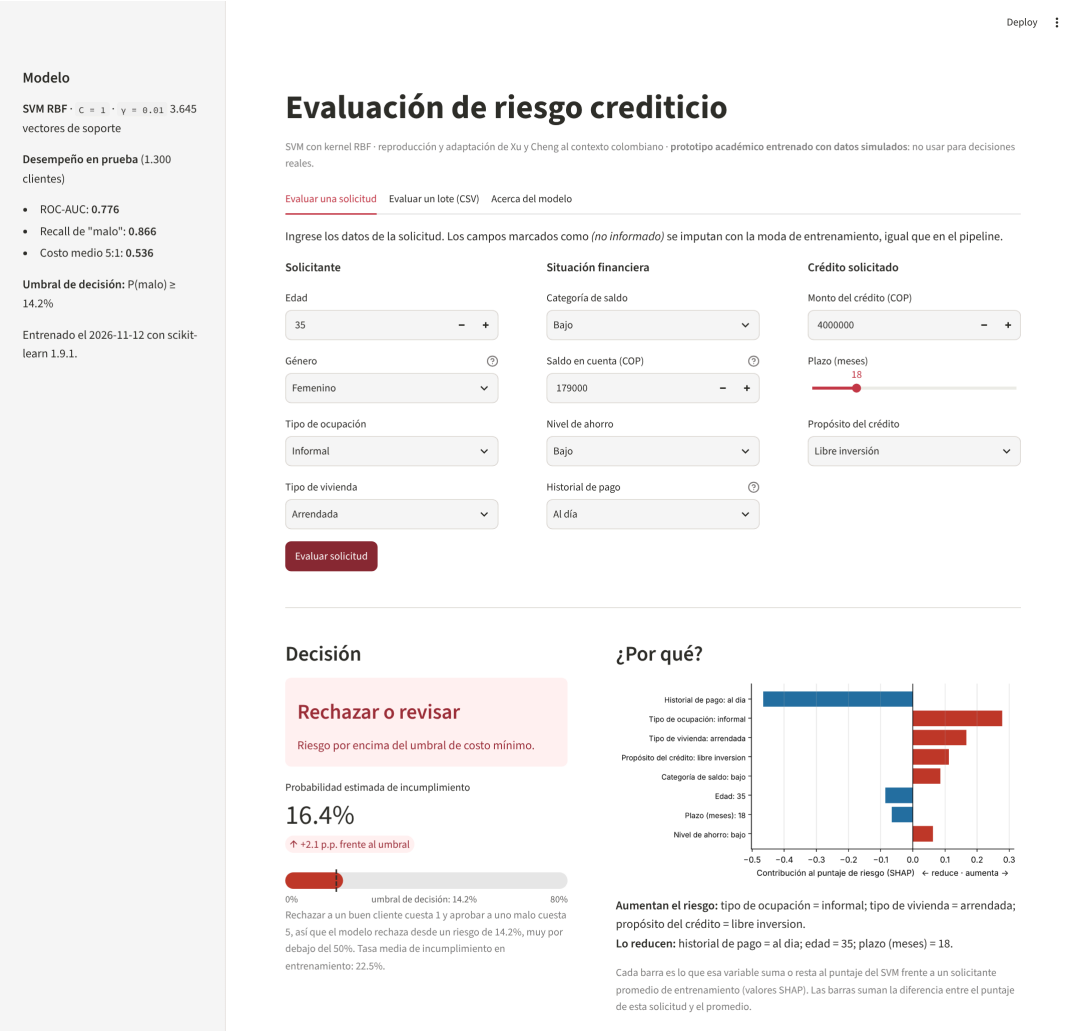


Figura 7. Evaluación de una solicitud: formulario, decisión con la probabilidad calibrada y el umbral, y explicación SHAP de la predicción.

Coherencia entre los datos de entrenamiento y los de uso. El módulo `src/interpretacion.py` guarda junto al modelo un perfil de los datos de entrenamiento (categorías, rangos, faltantes y distribución de cada variable) y compara con él los datos que llegan a la aplicación. Las reglas se dividen en tres niveles. Los **errores** impiden puntuar la fila: columnas ausentes, categorías no vistas, valores imposibles (montos o plazos no positivos, menores de edad) y contradicciones (categoría "sin cuenta" con saldo positivo, o cuenta con saldo cero). Las **advertencias** señalan que el modelo extrapola: valores fuera del rango de entrenamiento, faltantes en variables muy influyentes y deriva importante (índice de estabilidad poblacional, PSI ≥ 0.25). Las observaciones **informativas** registran los faltantes que el pipeline imputa y la deriva moderada. Verificamos la regla en el notebook 06: el conjunto de prueba pasa sin errores, y en un lote alterado (más informalidad, más mora, montos 60% mayores y errores de captura) se detectan la categoría inventada, las contradicciones y la deriva del monto (PSI 0.49). El archivo de ejemplo de la aplicación incluye cuatro filas con errores para demostrarlo (Figura 8).

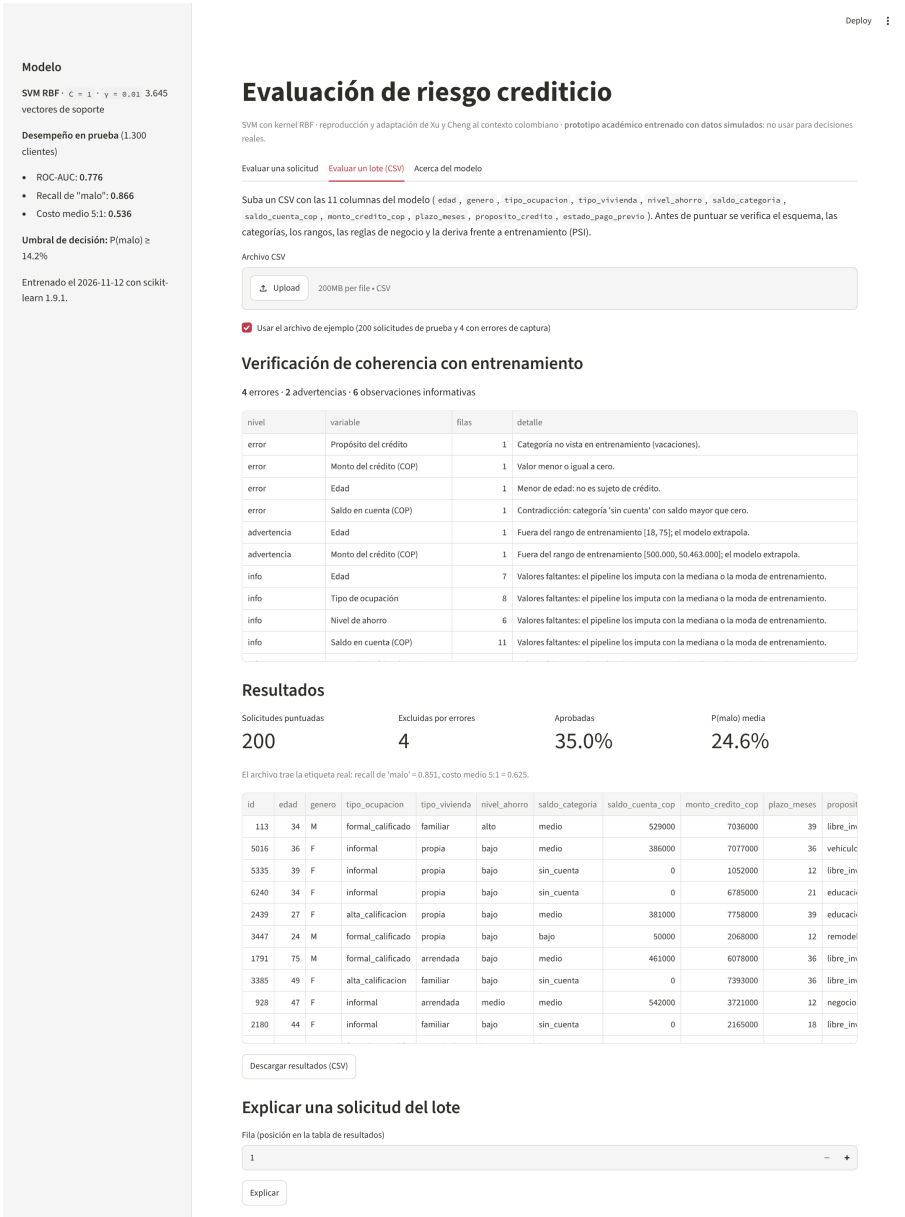


Figura 8. Evaluación de un lote: verificación de coherencia (errores, advertencias e información), resultados con la probabilidad de cada solicitud y descarga.

Cómo ejecutarla. Desde la raíz del repositorio, con las dependencias de `requirements.txt` instaladas: `python src/modelo_final.py` (entrena y guarda el modelo, unos 20 segundos; solo es necesario si el archivo no existe o si cambian las versiones de las librerías) y `streamlit run app/app.py`, que abre la aplicación en `http://localhost:8501`. El README del repositorio documenta ambos pasos.

4. Calidad, limitaciones y proyección

4.1 Limitaciones

- **Calidad y realismo de los datos.** Es la limitación principal. La base es simulada porque la información crediticia individual está protegida por la Ley 1266 de 2008. El riesgo se generó como función de los mismos predictores, así que los modelos se acercan a un techo conocido y los resultados no anticipan el desempeño con clientes reales.
- **Supuestos de la simulación.** La dirección del efecto de no tener cuenta y del historial de pago es una decisión de adaptación que contradice lo observado en el German Credit; la informalidad (44%), la tenencia de vivienda y la prevalencia de 22.5% son supuestos razonables pero no verificados dato a dato. La estructura casi lineal favorece a los modelos lineales y limita lo que el kernel RBF o los ensambles pueden mostrar.
- **Riesgo de sobreajuste.** Está controlado: la selección de hiperparámetros y del umbral se hizo dentro de la validación anidada, el conjunto de prueba se usó una sola vez y las métricas de prueba coinciden con las anidadas. Las curvas de validación mostraron que valores altos de γ sobreajustan, y el modelo elegido está lejos de esa zona.
- **Estabilidad.** Los hiperparámetros, el umbral y el ranking de importancia son estables entre folds, pero la incertidumbre de una sola muestra de prueba es grande (el intervalo de costo tiene un ancho de unas 0.10 unidades), mayor que las diferencias entre los mejores modelos.
- **Generalización.** El modelo solo conoce las 11 variables y los rangos de la simulación. Cualquier cambio de población (otra región, otro producto, un ciclo económico distinto) requiere reentrenarlo con datos reales. La matriz 5:1 también proviene de otro contexto.
- **Interpretación.** SHAP supone independencia entre variables al reemplazarlas por las del fondo, lo que explica el reparto entre variables correlacionadas; las explicaciones describen al modelo, no causalidad.

4.2 Mejoras y líneas de continuidad

- **Datos reales.** Validar el flujo con datos anonimizados de una entidad o de una central de riesgo, bajo un acuerdo de confidencialidad, e incorporar variables que hoy faltan: ingresos, endeudamiento total, comportamiento transaccional y pagos de servicios públicos (datos alternativos útiles para no bancarizados).
- **Política y equidad.** Definir la matriz de costos con la entidad, retirar el género, evaluar umbrales por segmento y monitorear métricas de equidad (tasas de aprobación y de rechazo de buenos clientes por grupo). Tratar el historial delgado con información alternativa en lugar de penalizarlo.
- **Modelo.** Explorar un modelo aditivo interpretable (por ejemplo, logística con términos no lineales en la edad) que combine la transparencia de la logística con la forma que captura el SVM.

4.3 Proyección hacia *Deep Learning* y MLOps

Deep Learning*.** Con las 11 variables tabulares actuales, una red neuronal no aportaría: el techo teórico ya está alcanzado. Su valor aparecería con datos más ricos: **redes recurrentes o *transformers sobre la secuencia de transacciones y pagos de cada cliente, que hoy resumimos en una sola categoría de

historial; **arquitecturas tabulares** (como TabNet o redes con *embeddings* de categóricas) cuando haya muchas variables de alta cardinalidad; y **autoencoders** para detectar solicitudes anómalas o posibles fraudes antes de puntuarlas. Cualquiera de ellos debería compararse contra el SVM y la logística con la misma validación anidada, y explicarse con las mismas herramientas.

MLOps. El despliegue actual ya contiene las piezas básicas: un pipeline único, un artefacto versionado con su perfil de entrenamiento y una verificación de datos de entrada. Los siguientes pasos serían: registrar experimentos y modelos (por ejemplo, con MLflow) y versionar los datos (DVC); empaquetar la aplicación en un contenedor y exponer el modelo como servicio; automatizar pruebas del pipeline, de la calibración y de la verificación de datos en integración continua; **monitorear en producción** la deriva de las variables (el PSI que ya calculamos), la tasa de aprobación y, cuando lleguen las etiquetas, el desempeño y la equidad; y definir reglas de reentrenamiento y una ficha del modelo que documente su uso y sus límites.

5. Conclusiones

Cerramos el flujo completo del proyecto. El modelo final es un **SVM con kernel RBF** que alcanza en prueba el máximo desempeño posible con estas variables (ROC-AUC 0.776, recall 0.866, costo 0.536), con probabilidades bien calibradas y una explicación fiel a la estructura verdadera de los datos. Reprodujimos el desempeño del método de Xu y Cheng, pero no su principal afirmación: el SVM no supera a una regresión logística preparada en igualdad de condiciones, ni en el German Credit ni en el contexto colombiano simulado, y la elección entre ambos depende más de la interpretabilidad requerida que del desempeño. El despliegue en Streamlit muestra que el modelo puede usarse de forma responsable: verifica sus datos de entrada, decide con una matriz de costos explícita y explica cada decisión. Las mayores mejoras posibles no vienen de un algoritmo más complejo sino de mejores datos y de decisiones de política explícitas sobre costos e inclusión.

Código y reproducibilidad

Todo el código está en el repositorio `github.com/danielaestudio02-del/svm_crediticio_col`. Además de lo descrito en la Segunda entrega, la Entrega final añade: `src/modelo_final.py` (entrena y guarda el modelo final), `src/interpretacion.py` (explicaciones SHAP y verificación de los datos de uso), `notebooks/06_Modelo_final_interpretacion.ipynb` (Secciones 1 y 2 de este informe), `app/app.py` (despliegue) e `informe/generar_informe_final.py`, que regenera este PDF. Todo usa semilla fija (42) y versiones de librerías fijadas en `requirements.txt`.

Uso de inteligencia artificial generativa

El equipo utilizó herramientas de inteligencia artificial generativa como apoyo en la escritura y revisión del código, la depuración y la redacción. Todas las decisiones metodológicas y los resultados fueron revisados por el equipo, que puede sustentarlos.

Referencias

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189-1232.

Goldstein, A., Kapelner, A., Bleich, J. y Pitkin, E. (2015). Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics, 24*(1), 44-65.

Hofmann, H. (1994). Statlog (German Credit Data) [Conjunto de datos]. UCI Machine Learning Repository.

Ley 1266 de 2008. Por la cual se dictan las disposiciones generales del hábeas data. Congreso de la República de Colombia.

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*.

Nadeau, C. y Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*(3), 239-281.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. En *Advances in Large Margin Classifiers* (pp. 61-74). MIT Press.

Ribeiro, M. T., Singh, S. y Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.

Xu, J. y Cheng, Y. (s. f.). *Credit scoring models enhancement using support vector machines*.